

Lab1:DataLab

```
./check_ops.py bit.c
```

```
(base) waldein@waldein-OMEN-Gaming-Laptop-16-am0xxx:~/25300180063lab2$ ./check_ops.py /home/waldein/25300180063lab2/bits.c
signMask: 2/2 operators
bitXor: 8/8 operators
negativePart: 4/6 operators
copyByteWithin: 10/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 18/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 31/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 11/24 operators
midpointTowardFirst: 27/32 operators
isBetweenEitherOrder: 28/48 operators
mul5Sat: 23/30 operators
classifyAdd3: 35/52 operators
floatScaleThreeHalves: 43/60 operators
floatRoundEven: 39/65 operators
float_i2f: 26/40 operators
bitCount: 36/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

```
./btest
```

Score	Rating	Errors	Function
1	1	0	signMask
2	2	0	bitXor
3	3	0	negativePart
4	4	0	copyByteWithin
4	4	0	logicalShift
4	4	0	swapNibblePairs
4	4	0	secondLowestZeroBit
5	5	0	oddParity
5	5	0	rotateRightBits
5	5	0	roundEvenPow2
5	5	0	midpointTowardFirst
7	7	0	isBetweenEitherOrder
7	7	0	mul5Sat
7	7	0	classifyAdd3
7	7	0	floatScaleThreeHalves
10	10	0	floatRoundEven
10	10	0	float_i2f
10	10	0	bitCount
10	10	0	bitReverse

Total points: 110/110

思路:

P1:

要求输出0x80000000, 利用~0=0xFFFFFFFF, 左移31位即可

P2:

用~和&实现^, 要求符合 $1 \wedge 1 = 0$, $0 \wedge 0 = 0$, $1 \wedge 0 = 1$, $0 \wedge 1 = 1$, 以此设计即可

P3:

要求 $x < 0$ 时, 返回 $-x$, otherwise返回0. 可以先做 $s = x >> 31$, 利用符号位, $x < 0$, 得到 $0xFFFFFFFF$; $x \geq 0$, 得到 $0x00000000$. 然后 $s \& x$ 再取反再+1, $x < 0$, 恰好为常规的负数取绝对值; $x \geq 0$, 先得到 $0xFFFFFFFF$, 再+1得到0

P4:

复制粘贴byte. 先将src对应byte取出并移位到byte0, 再扣除dst对应byte, 再将source移位到destination, 和之前扣除后的结果做|运算, 得到答案.

P5:

逻辑移位. 先算术移位, 再利用&运算, 去除负数移位产生的1, 得到答案.

P6:

先构造 $0x0000FFFF$ 和 $0xFFFF0000$, 然后取&, 移位, 取|, 实现交换, 得到答案.

P7:

找0出现的第二小数, 相当于取~后, 找1出现的第二小数. 先取逆, 加上 (~ 0) , 相当于-1, 将最靠右的1向右移一个bit, 再&运算上原来的数, 去除最靠右的1. 得到新的数z确定它右的1的位置即可, 取 $z \& ((\sim z) + 1)$, 可以得到所需结果

P8:

注意到, 对于输入的数x, 设其左16位有a个1, 右16位有b个1, 让左16位和右16位做异或^, 结果中1的数量为 $a+b-2n$, 奇偶性不变, 类似做下去, 直到只剩一位数c, $c = a+b \pmod{2}$, 为需要的结果.

P9:

和之前的左16位和右16位调换基本类似

P10:

以8bits数为例, $x_1 = 01111000$, $x_2 = 01101000$, $n = 4$. 注意 $2^{*4} = 00010000$, 要按题目要求近似, 考虑让x先加上 00000111 , 保证非恰好处于两数中间时, 可以近似到离它更近的数, 此时 $x_1 = 01111111$, $x_2 = 01101111$, 注意到要求的"奇偶"判定取决于第n位(从第0位数起)的值, 单独取出判定, 考虑是否对数+1, 然后舍弃右n位即可

P11:

确定 x 和 y 符号, $(x+y)/2$ 的计算, 采用 $0.5x+0.5y$, 需要舍入, 这里的舍入采用了较为复杂的逻辑, 目的是对于 x 奇 y 偶, x 奇 y 奇, x 偶 y 奇, x 偶 y 偶, 以及 x 正 y 正, x 负 y 正, x 正 y 负, x 负 y 负, 的情况, 采用正确的舍入方法

P12:

判断 x 是否在 a, b 间. 确定 $(x-a)(x-b)$ 的符号即可, 由于可能发生溢出, 需要先尽可能排除溢出的情况, 溢出仅当 x 和 a (or b)异号, 这种情况不需要做减法就可以判断 x 是否在 a, b 间, 于是可以先用 x, a, b 的符号进行判断, 然后判断 $(x-a)$ 和 $(x-b)$ 的符号, 最后将 $x=a$ 或 $x=b$ 的特殊情况考虑进来即可

P13:

$x*5$, 就是 $(x<<2)+x$, 需要依次判断, $x<<1, x<<2, (x<<2)+x$, 是否发生溢出, 只要有一个发生溢出, 就判定为溢出, 然后根据正负性对于溢出的情况特殊处理

P14:

判断 $x, y, z, x+y, x+y+z$ 的符号, 根据异号相加不溢出, 同号相加可能溢出, 也可能修正之前的溢出, 三个同号数相加可能二次溢出, 一次溢出必变号. 这些引理, 然后, 先判断所有的可能性, 再按是否溢出和是否二次溢出分类, 转译为代码, 最后根据 $x+y+z$ 的符号以及溢出的判断结果, 区分正溢出和负溢出, 再根据逻辑判断确定返回结果

P15:

先单独处理特殊情况, 再将规格化和非规格化数各自处理成实际表示的数, 乘3再除2, 确定结果的指数大小并做舍入

P16:

单独处理特殊情况, 对于整数和较小的数, 可以利用 exp 直接判断舍入结果并返回, 对于其余情况, 判断整数部分奇偶, 按实际小数位结果, 决定输出

P17:

先处理0和 int_min . 对于其他情况, 取出符号位, 再利用左移找到最大位, 暂时确定 exp , 然后对长度超过23位的小数部分做舍入, 并判断舍入后 exp 是否发生变化, 修正 exp 和 frac , 组合返回结果

P18:

32位数 x , 分为8个byte, 构造工具 $t=0x11111111$, 通过 $\&$ 和右移 x , 4次重复可以检测完所有的byte, 并将结果加到暂时的 result 中, 对每个byte分别及计结果, 比如 $x=0xFEDC4321, \text{result}=0$, 处理后 $\text{result}=0x43321211$, 然后将每个byte的结果相加, 返回1的个数

P19:

对32位的数,分成前16位和后16位两组,交换,交换后对这两组按类似的方法各自对半分为两个长为8位的小组,前16位的两个8位小组相互交换,后16的两个8位小组相互交换,然后再按同样方法分组并交换,一直做下去直到不能再分组,实现逆序